COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	AUGUST 4 2026	**DATE SUBMITTED:**	AUGUST 5 2026
NAME:	DUSTIN CARL B. MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 5.1 **HANDS-ON ACTIVITY 5.1: QUEUES**

1. PROCEDURE OUTPUT

ILOA

Code

main.cpp > main()

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main(){
    //create an object
    std::queue<char>myQ;

    myQ.push('C');
    myQ.push('A');
    myQ.push('R');
    myQ.push('L');
    std::cout<<"Displaying Queue: "<<std::endl;
    display(myQ);
    //try all member function
    //empty
    std::cout<<"Is the queue empty?: "<<myQ.empty()<<std::endl;
    //size
    std::cout<<"The size of the queue: "<<myQ.size()<<std::endl;
    //front
    std::cout<<"What is the front of the queue?: "<<myQ.front()<<std::endl;
    //back
    std::cout<<"What is the rear of the queue?: "<<myQ.back()<<std::endl;
```

```
24      std::cout<<"What is the rear of the queue?: "<<myQ.back()<<std::endl;
25      //pop
26      std::cout<<"Popping Queue: "<<std::endl;
27      myQ.pop();
28      //display
29      std::cout<<"Displaying Queue: "<<std::endl;
30      display(myQ);
31      //push
32      std::cout<<"Pushing '!': "<<std::endl;
33      myQ.push('!');
34      //display
35      std::cout<<"Displaying Queue: "<<std::endl;
36      display(myQ);
37
38      return 0;
39  }
40  void display(std::queue<char> copyQ){
41      //creates a copy of he queue that was passed
42      std::queue<char> c = copyQ;
43      while(!c.empty()){
44          std::cout<<" "<<c.front();
45          c.pop();
46      }
47      std::cout<<"\n";
48  }
```

Output

```
[Running] cd "c:\Users\TIPQC\Documents\CPE_M
Displaying Queue:
 C A R L
Is the queue empty?: 0
The size of the queue: 4
What is the front of the queue?: C
What is the rear of the queue?: L
Popping Queue:
Displaying Queue:
 A R L
Pushing '!':
Displaying Queue:
 A R L !

[Done] exited with code=0 in 0.503 seconds
```

Analysis of ILOA

In this task, I created a queue STL implementation. I first include the libraries iostream and queue. After that I made a function prototype for displaying all the elements in the queue and then defined that function below the main function. In the display function I did it by copying the created queue and then used a while loop for displaying Moving on to the main function, I created an object named “myQ” I then pushed 4 characters using the push method and displayed it. After that I tried all the member functions which are empty() where it checks if the queue is empty, size() where it checks the size of the queue, front() wherein it checks the data that is in the front, and then back where it checks the data that is in the back. Next is pop() where it pops the first element, and then push where it pushes a character in the queue.

ILOB.1

Header File

queueLL.h > ...

```h
#ifndef QUEUELL_H
#define QUEUELL_H

//create a class the node
template <typename T>
class qNode{
    public:
        T data;
        qNode<T>* next;
};

//create the new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

//enqueue operation
template<typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    //create the new node
    qNode<T>* newNode = new_node(newData);

    //inserting into an empty queue
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }
    //inserting an item into a non empty queue
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}
```

```
36
37  //dequeue operation
38  template<typename T>
39  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
40      //craete a temporary node
41      qNode<T>* deleteNode = nullptr; //temp variable
42      deleteNode = (*frontPtr);
43
44      //check if the queue has only 1 node
45      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
46          (*frontPtr) = nullptr;
47          (*backPtr) = nullptr;
48          delete deleteNode;
49          return;
50      }
51      //delete the node
52      (*frontPtr) = (*frontPtr)->next;
53      deleteNode->next = nullptr;
54      delete deleteNode;
55
56  }
57
```

```
58  // check the front
59  template<typename T>
60  void frontCheck(qNode<T>** frontPtr){
61      if((*frontPtr) == nullptr) return;
62      std::cout<<"Front is: "<<(*frontPtr)->data<<"\n";
63  }
64  // check the back
65  template<typename T>
66  void backCheck(qNode<T>** backPtr){
67      if((*backPtr) == nullptr) return;
68      std::cout<<"Back is: "<<(*backPtr)->data<<"\n";
69  }
70
71  //display all
72  template<typename T>
73  void displayAll(qNode<T>** frontPtr){
74      qNode<T>* temp = *frontPtr;
75      std::cout<<"The Data are: "<<std::endl;
76      if((*frontPtr) == nullptr) return;
77      while(temp != nullptr){
78          std::cout<<temp->data<<" ";
79          temp = temp->next;
80      }
81      std::cout<< std::endl;
82  }
83
84  #endif
```

Main

main.cpp > main()

```cpp
#include <iostream>
#include "queueLL.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;


    enqueue('C', &front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue('A', &front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;
    enqueue('R', &front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;

    dequeue(&front, &back);
    std::cout<<front->data<<" "<<back->data<<std::endl;

    frontCheck(&front);
    backCheck(&back);
    displayAll(&front);

    return 0;
}
```

Output

```
[Running] cd "c:\Users\TIPQC\Documents\CPE
C C
C A
C R
A R
Front is: A
Back is: R
The Data are:
A R

[Done] exited with code=0 in 0.42 seconds
```

Analysis in ILOB.1

I first created the header file called “queueLL.h”. After that, I used #ifndef and #define. Next I created a class node, and then I created a new node. After that I made the enqueue operation, this operation pushes data inside the queue. Next I created the dequeue operation where the program will delete the front of the queue. Next is the frontCheck() function where it will check or display the data in the front. If the front is null, it will just return. Furthermore, the backCheck function checks the back on the queue and displays the data inside it. Lastly, the displayAll() function where it will display all the data inside the queue. I did this by creating a temporary pointer and storing the front pointer to it. After that, I used a while loop that will continuously print the data inside the queue when temp is not equal to nullptr.

ILOB.2

Header file

```cpp
#ifndef QUEUEARR_H
#define QUEUEARR_H

class Queue {
    private:
    char* q_array;
    int q_capacity;
    int q_size;
    int q_front;
    int q_back;
    public:
    // Constructor
    Queue(int size = 6) {
        q_capacity = size;
        q_array = new char[q_capacity];
        q_size = 0;
        q_front = 0;
        q_back = -1;
    }
    // Deconstructor
    ~Queue() {
        delete[] q_array;
    }
    // Copy Constructor
    Queue(const Queue& other) {
        q_capacity = other.q_capacity;
        q_array = new char[q_capacity];
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;
        for (int i = 0; i < q_capacity; i++) {
            q_array[i] = other.q_array[i];
        }
    }
```

```
35      // Copy Assignment Operator
36      Queue& operator=(const Queue& other) {
37          if (this != &other) {
38              delete[] q_array;
39              q_capacity = other.q_capacity;
40              q_array = new char[q_capacity];
41              q_size = other.q_size;
42              q_front = other.q_front;
43              q_back = other.q_back;
44              for (int i = 0; i < q_capacity; i++) {
45                  q_array[i] = other.q_array[i];
46              }
47          }
48          return *this;
49      }
50
51      bool empty() const {
52          return (q_size == 0);
53      }
54
55      int size() const {
56          return q_size;
57      }
58
59      void clear() {
60          q_size = 0;
61          q_front = 0;
62          q_back = -1;
63      }
64
```

```
65      char front() const {
66          if (empty()) {
67              std::cout << "Queue is empty" << std::endl;
68              return '\0';
69          }
70          return q_array[q_front];
71      }
72
73      char back() const {
74          if (empty()) {
75              std::cout << "Queue is empty" << std::endl;
76              return '\0';
77          }
78          return q_array[q_back];
79      }
80
81      void enqueue(char value) {
82          if (q_size == q_capacity) {
83              std::cout << "Queue is full" << std::endl;
84              return;
85          }
86          q_back = (q_back + 1) % q_capacity;
87          q_array[q_back] = value;
88          q_size++;
89          std::cout << "Enqueued: " << value << " \n";
90      }
91
92      void dequeue() {
93          if (empty()) {
94              std::cout << "Queue is empty" << std::endl;
95              return;
96          }
97          std::cout << "Dequeued: " << q_array[q_front]<<std::endl;
98          q_front = (q_front + 1) % q_capacity;
99          q_size--;
100     }
101
```

```
102     void display() const {
103         if (empty()) {
104             std::cout << "Queue is empty" << std::endl;
105             return;
106         }
107         std::cout << "The Queue (front to back): ";
108         for (int i = 0; i < q_size; i++) {
109             int index = (q_front + i) % q_capacity;
110             std::cout << q_array[index] << " ";
111         }
112         std::cout << std::endl;
113     }
114
115 };
116
117 #endif #ifndef QUEUEARR_H
```

Code

```cpp
int main() {
    std::cout << "Queue (Array Implementation)" << std::endl;
    Queue qT( size: 6);

    std::cout << "Enqueueing..." << std::endl;
    qT.enqueue( value: 'D');
    qT.enqueue( value: 'U');
    qT.enqueue( value: 'S');
    qT.enqueue( value: 'T');
    qT.enqueue( value: 'I');
    qT.enqueue( value: 'N');

    std::cout << "Displaying Queue:\n ";
    qT.display();

    std::cout << "Front element: " << qT.front() << "\n";
    std::cout << "Back element:  " << qT.back() << "\n";
    std::cout << "Current size:  " << qT.size() << "\n";

    std::cout << "Dequeue Operation\n";
    qT.dequeue();
    qT.dequeue();
    qT.display();

    std::cout << "Enqueueing...\n";
    qT.enqueue( value: 'G');
    qT.enqueue( value: 'E');
    qT.display();
```

```
144
145     std::cout << "Copy Constructor\n";
146     Queue qP = qT;
147     std::cout << "Original (qT): ";
148     qT.display();
149     std::cout << "Copied (qP):   ";
150     qP.display();
151
152     std::cout << "Testing Clear\n";
153     qT.clear();
154     std::cout << "Is qT empty after clear()? " << (qT.empty() ? "Yes" : "No") << "\n";
155     qT.display();
156
157     std::cout << "Verifying Deep Copy\n";
158     std::cout << "qP: ";
159     qP.display();
160
161
162     return 0;
163 }
```

Output

```
"C:\Users\Dread\Desktop\cpe 010\HOA5p1\ILOB2\mainB2.exe
Queue (Array Implementation)
Enqueueing...
Enqueued: D
Enqueued: U
Enqueued: S
Enqueued: T
Enqueued: I
Enqueued: N
Displaying Queue:
 The Queue (front to back): D U S T I N
Front element: D
Back element:  N
Current size:  6
Dequeue Operation
Dequeued: D
Dequeued: U
The Queue (front to back): S T I N
Enqueueing...
Enqueued: G
Enqueued: E
The Queue (front to back): S T I N G E
Copy Constructor
Original (qT): The Queue (front to back): S T I N G E
Copied (qP):   The Queue (front to back): S T I N G E
Testing Clear
Is qT empty after clear()? Yes
Queue is empty
Verifying Deep Copy
qP: The Queue (front to back): S T I N G E

Process finished with exit code 0
```

Code Analysis for ILOB.2

I first created a header file for the array implementation of queue called “queueArr.h”. And there I created a class called Queue, wherein the private variables are the array, the capacity, the size, the front,and the back. I also created a constructor, and the big three which are destructor, copy constructor, and copy assignment operator. Next is that I defined every function that can be used for a queue like empty() where it checks if the queue is

empty, size() wherein it returns the size of the queue, clear(), it resets the queue, front() displays the front of the queue. Moreover there is also back() where it returns the data in the back of the queue. enqueue() where it adds data into the queue, and dequeue() where it removes the front of the queue. Lastly, display() where it displays the data in the queue. Moving on to the main file, I declared a variable called qT with a size of 6, and then I enqueued characters, after that I displayed it using the display function. Furthermore, I displayed the font, back, and size of the queue. I also dequeued, I also used the copy constructor, and tested if the deep copy worked which it did.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILOC

Header File (Check ILOB.1 for header file code screenshot)

Code

```
#include <iostream>
#include "queueLL.h"
#include <string>

class Job {
public:
    int id;
    std::string userName;
    int pages;
};

class Printer {
private:
    qNode<Job>* frontPtr = nullptr;
    qNode<Job>* backPtr = nullptr;

public:
    void addJob(int id, std::string userName, int pages) {
        Job j = {id, userName, pages};
        enqueue(j, &frontPtr, &backPtr);
        std::cout << "Adding job " << id <<" for "  << userName<<std::endl;
    }

    void processJob() {
        std::cout << "\n Processing... \n";
        while (frontPtr != nullptr) {
            Job j = frontPtr->data;
            std::cout << "Printing Job " << j.id << " (" << j.userName << ", " << j.pages << " pages)...\n";
            dequeue(&frontPtr, &backPtr);
        }
        std::cout << "Jobs Completed!\n";
    }

};
```

```cpp
int main() {
    Printer printer;

    printer.addJob( id: 001,  userName:  "Moraine",  pages: 1);
    printer.addJob( id: 002,  userName:  "Rand",  pages: 5);
    printer.addJob( id: 003,  userName:  "Egwene",  pages: 3);
    printer.addJob( id: 004,  userName:  "Siuan",  pages: 1);
    printer.addJob( id: 005,  userName:  "Nyneave",  pages: 1);

    printer.processJob();

    return 0;

}
```

Output

```
"C:\Users\Dread\Desktop\cpe 010\HOA5p1
Adding job 1 for Moraine
Adding job 2 for Rand
Adding job 3 for Egwene
Adding job 4 for Siuan
Adding job 5 for Nyneave

 Processing...
Printing Job 1 (Moraine, 1 pages)...
Printing Job 2 (Rand, 5 pages)...
Printing Job 3 (Egwene, 3 pages)...
Printing Job 4 (Siuan, 1 pages)...
Printing Job 5 (Nyneave, 1 pages)...
Jobs Completed!

Process finished with exit code 0
```

Code Analysis for ILOC

In this supplementary activity, I used the queue linked list implementation header file that was done on the procedure ILOB.1. Moreover, I also included iostream and string for this program. After that, I created the Job class wherein it has public variables and this is where the id, username of a person, and the pages of their printing material is stored. Next I created the Printer class wherein it has different functions like addJob() and processJob() where in addJob(), it adds a printing job for the program to add in the queue, while the processJob() processes the added job and display the printing jobs. Now moving on to the main function, I created a variable printer with the Printer type and then I used that variable and the dot operator to access all the functions needed to print. With that, I used a linked list implementation because it is much easier to do, and that I already have created a header file for it. Lastly, linked list implementation because there is no size limit, and as soon as the printing job is finished it will delete the unused nodes thus freeing memory.

Conclusion:

In this activity, I learned about the definition of queue, and how I can implement it in coding. I also learned about using the queue STL, and also the linked lists, and array implementation of a queue. At last, I also did an analysis for each procedure and supplementary activity. And that this activity was fun and I understood the lessons, but I still need to improve my programming skills.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Concept Explanations, and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ai so that I can understand concepts much better and also for guidance.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.